

LLM-Guided Distillation for Real-Time End-to-End Driving in a Simulated Environment

Abstract

1 Introduction

Autonomous driving tasks typically require extensive, high quality datasets along with precise human supervision. Collecting data to build a autonomous driving system in real life is expensive and raises safety concerns. Modern Large Language Models (LLMs) have been shown to be quite capable at reasoning about visual scenes, traffic rules, and driving behavior [8][10].

Grand Theft Auto V, with its extensive map [], provides such an environment that enables us to really put this idea to test. In this work, we investigate whether LLMs can serve as “teachers” that label driving behavior in GTA V, enabling training of a compact student model that runs in real time on consumer hardware.

The core question we address is: *Can we distill an LLM’s high-level driving intuition into a small vision model that outputs low-level controls in real time?* [10]

Our main contributions are:

- We propose a pipeline in which an LLM labels GTA V frames with low-level driving controls (steering, throttle, brake).
- We design and train a lightweight n-frame vision student (based on EffecientNet) that models the LLM’s behavior and runs in real time on an RTX 2060-class GPU.
- We implement a complete real-time deployment loop that captures frames from GTA V, feeds them to the student, and applies controls to the game.
- We present quantitative and qualitative evaluations comparing highway vs. city scenarios, and various training choices.

2 Related Work

2.1 End-to-End Driving

Prior work on end-to-end driving has explored convolutional networks trained from human demonstrations, as well as large-scale systems deployed in real vehicles. GTA V has also been used as a sandbox for learning-based driving policies [25][23].

2.2 Vision Backbones for Driving

Compact convolutional networks such as MobileNet and EfficientNet, and more recent transformer-based backbones such as ViT and FastViT, provide strong image representations under tight latency constraints, making them suitable for driving policies.

2.3 LLMs and Vision-Language Reasoning

Vision-capable LLMs have shown strong performance on scene understanding, reasoning about traffic, and following natural-language instructions, suggesting they may be useful as supervision sources for downstream models [1][2][3].

2.4 Knowledge Distillation

Knowledge distillation and policy distillation frameworks train smaller student models to mimic larger teachers, often improving efficiency while retaining much of the teacher’s performance. Our work applies this idea to LLM-guided driving.

3 Methodology

3.1 System Overview

Our system consists of four main components:

1. **Data collection:** record gameplay from GTA V while driving in various scenarios.
2. **LLM teacher:** for each frame (or short temporal window), the LLM predicts low-level controls with a explanation of its reasoning.
3. **Vision student:** a lightweight 3-frame model trained to regress the teacher’s controls.
4. **Real-time deployment:** a loop that captures frames, runs the student, and applies controls in the game.

3.2 Data Collection in GTA V

We recorded footage in GTA V at 854×480 at 30 FPS in windowed mode. FFMPEG was used to split the clips into frames; together we recorded 18 sessions totaling 50,000 frames. The recorded footage was in diverse environments including nighttime and daytime sessions with a variety of different conditions such as rainy, cloudy, and sunny conditions to ensure that the model would be fully exposed to the unknowns of real world driving.

3.3 Pre-processing

A screen grab of GTA V game play has a lot of UI elements such as the HUD which includes the mini map and any popup notifications that may come up in game play. These may prove quite problematic when training vision models as they might focus on those elements instead of extracting the necessary features in a scene, such as lane lines, obstacles, and traffic signals. To combat this and reduce the amount of noise in a given frame, we mask the mini-map with a rectangle with a nearby color. This ensures that we do not lose any valuable scene data if we were to crop it and remove any hud content the model could memorize. All frames are resized using the following operations:

```
T.Resize((input_size , input_size))
T.ToTensor()
T.Normalize(IMNET_MEAN, IMNET_STD)
```

We use the standard image net normalization statistics with the Mean: [0.485, 0.456, 0.406] and Std: [0.229, 0.224, 0.225].

3.3.1 Data Augmentation

To increase robustness and prevent overfitting to specific lighting or slight camera variations, two types of augmentations are applied only during training:

3.3.2 Color Jitter

Parameters:

- Brightness ± 30
- Contrast ± 30
- Saturation ± 20
- Hue ± 0.05

These augmentations simulate weather sudden lighting changes such as going into a tunnel or being flashed by a high beam by an oncoming car to help the model better generalize the environment. These parameters are applied consistently to the 3 frames frames in the 3-frame model to ensure temporal understanding.

3.3.3 Random Affine Transformations

Parameters:

- Rotations ($\pm 7^\circ$)
- Translations ($\pm 5\%$)
- Perspective shifts

These augmentations are applied to simulate natural camera variations that can occur while the vehicle is in motion or by the steering input that is applied by the driver. Similar to the color jitter parameters, these augmentations are applied consistently.

3.4 LLM Teacher Labeling

3.4.1 Motivation for LLM Supervision

State-of-the-art Large Language Models have great scene understanding capabilities, including the ability to interpret lane lines, traffic signs, and traffic rules. For example, if shown an image of a car that is drifting out of its lane the LLM may infer the corrective action needed is to apply steering to recenter the vehicle. by leveraging this strong reasoning ability we negate the need for human annotated driving actions, dramatically reducing the time and effort needed to create a dataset sufficient to model the driving environment [4][5][6][7].

3.4.2 Prompt Design and Output Format

The LLM prompt seen below is instructed to act like an expert human driver, it is fed in 3 consecutive frames and is instructed to output low level driving instructions about what a human driver would do in the middle frame. Supplying the LLM with 3 frames allows the LLM to infer motion and vehicle dynamics, thus preserving the temporal context that is needed for driving.

Range constraints:

- *steer* $\in [-1, 1]$: -1 steer to the left and 1 is steer to the right
- *throttle* $\in [0, 1]$: 0 is no throttle and 1 is full throttle
- *brake* $\in [0, 1]$: 0 in no brake and 1 is full brake

We also include "interpret steering relative to the car's forward motion", this was added because the LLM would give incorrect commands such as go straight when the car was making a turn. This was because the LLM was not looking at the scene from the perspective of the car rather the overhead camera view it was given. We also added "reason_short" to better interpret what the model was seeing and allowed us to debug the teacher labels. From those iterations, we came up with the prompt below:

```
You're an expert human driver inside a realistic driving simulator.
You will be shown one or more consecutive frames from the FRONT camera.
Based only on what you can see, output a JSON object describing
what a safe and skilled human driver would do RIGHT NOW for the
MIDDLE frame. Respond with ONLY valid JSON (no explanations
outside the JSON). IMPORTANT CLARIFICATION: interpret steering
relative to the car's forward motion (NOT the camera/view perspective).
In your JSON: 'steer' in [-1,1] (-1=full LEFT, 0=straight,
+1=full RIGHT); 'throttle' in [0,1] (0=no gas, 1=full acceleration);
'brake' in [0,1] (0=no brake, 1=full brake);
```



Figure 1: GTA V Frame

'reason_short' is one concise sentence explaining your choice.

OUTPUT ONLY JSON, e.g.

```
{ "steer": -0.2, "throttle": 0.4, "brake": 0.0,
  "reason_short": "Turning left through intersection on wet road." }
```

Below is a real example of the label for Figure 1. This was generated by GPT 5.1 Mini

```
{
  "steer": -0.25,
  "throttle": 0.35,
  "brake": 0.0,
  "reason_short": "Gently steer left to follow the upcoming curve,
    keeping moderate throttle to maintain lane and speed."
}
```

3.4.3 Single-Frame vs Multi-Frame Reasoning

The main limitation with single frame labeling is that we are unable to perceive motion, to combat this we switched to a 3 consecutive frame approach, in which we are able to dramatically improve perception of movement, handling of curves, and traffic light inference. Temporal context = better teacher labels [18].

3.4.4 Label Quality Control

To ensure we have clean data to train on we must perform quality control on the teacher labels. We have devised a methodology to detect sudden changes in steering, braking, or throttle from one frame to another. This massively reduces the amount of human validation needed on the teacher labels, and we can focus on remediating labels by relabeling.

3.4.5 Teacher Limitations

As with any LLM occasional hallucinations are to be expected and can cause unexpected results in the downstream student. LLMs are non deterministic by nature and the same frame can result

in wildly different results creating data that will be difficult for the student model to generalize. The teacher also did not have context of previous frames outside of 3 we provided and caused the teacher to lose sight of the overall context of the scene, such as if a car is to the side of us out of camera or not. These problems caused inconsistent labeling and rather poor understanding of some scenes which propagated to the student model.

3.5 Student Model Architecture

To distill the information we learned from the teacher into a real time system that will run in the simulated environment, we designed a custom end-to-end vision model [12] built upon the EfficientNet-B2 vision backbone [15]. This model augments EfficientNet to receive multi frame inputs to preserve temporal context, spatial attention layers, along with decoupled lateral and longitudinal heads for steering and pedal prediction.

3.5.1 Backbone: EfficientNet-B2

We used EfficientNet-B2 as our backbone for our student policy, chosen mainly for its high performance with low computational overhead which is important for building safety critical systems such as autonomous vehicles.

The model operates on stacked temporal inputs, meaning we stacked 3 frames so 9 channels. However, EfficientNet only takes in 3 channels so we modified the first convolution layer to accept an n-channel convolution [19]. Using a pretrained model like EfficientNet provides us a strong base to learn more fine grained details in an image to make the car drive instead of learning from scratch the important features in an image.

3.5.2 Spatial Attention

When making driving decisions, we heavily depend on where we are looking at, such as the car in front of us to modulate our speed, the lane lines to make sure we are steering correctly.

To encourage the backbone to not look at the sky or other irrelevant regions in the frame, we introduce a spatial attention block which we apply to the feature map of the backbone [21][17].

$$A = \sigma(\text{Conv1} \times 1(\text{SiLU}(\text{Conv1} \times 1(F))))$$

- F is the original feature map
- A is a learned attention mask
- Features are modulated via $F' = F \odot A$

The block has a negligible compute overhead but significantly improves steering by highlighting the critical lane structure.

3.5.3 Global Feature Embedding

After the spatial attention layer, we feed the feature map through

- Adaptive average pooling
- Flattening

This results in a compact feature vector that provides an accurate representation of the driving scene at the moment.

3.5.4 Steering Head

The steering head [20] is designed to provide fine-grained curvature estimates and includes

- Three fully connected layers
- LayerNorm after each layer
- SiLU (Swish) activations to match EfficientNet
- Dropout for regularization
- A skip connection from the input embedding

Formally:

$$s = MLP(z) + W_{skip}z$$
$$steer = \tanh(W_s s)$$

The tanh enables the steering to be constrained to $[-1, 1]$. The residual connection enables better learning and gradient flow, especially when considering the unpredictable and noisy labels that LLMs may produce.

3.5.5 Pedal Head

The pedal head predicts:

- $Throttle \in [0, 1]$
- $Brake \in [0, 1]$

It is implemented with a deeper MLP:

- Two hidden layers ($512 \rightarrow 256$)
- LayerNorm
- SiLU activations
- Dropout

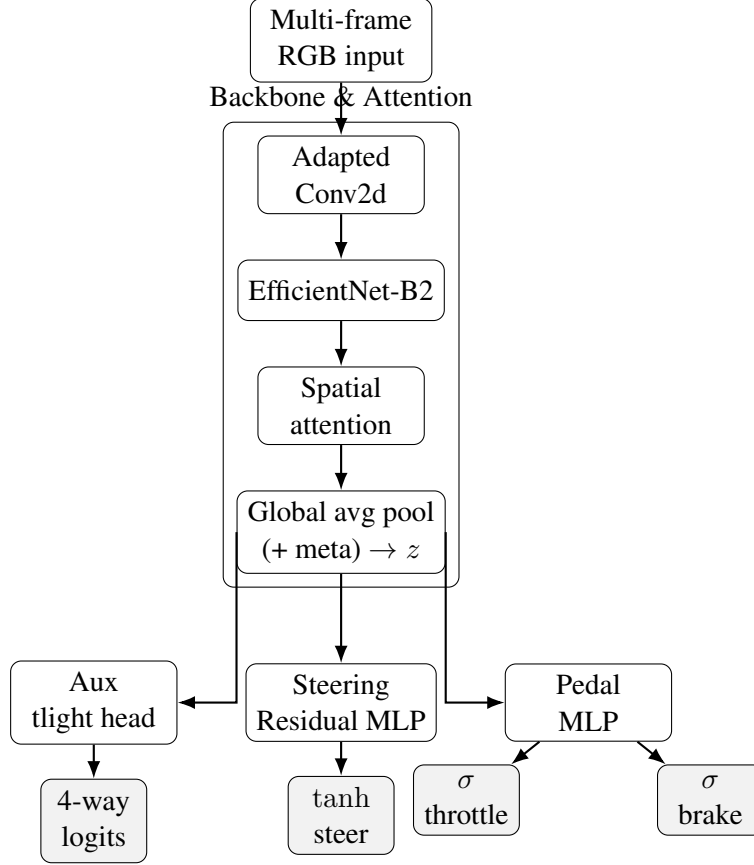


Figure 2: Compact student policy architecture. Multi-frame RGB input is processed by an EfficientNet-B2 backbone with spatial attention. Global pooled features form embedding z , which feeds decoupled steering, pedal, and optional traffic-light heads.

- A final linear layer producing two logits followed by sigmoid activations

The pedal head is larger to allow the model to reason about stopping distance, oncoming vehicles (e.g., swerving, hard braking), and traffic lights among others. Separation of the two heads allows the model to learn specific lateral or longitudinal control tasks.

3.6 Real-Time Agent Runner

To test the student model, we created a python script that closes the loop between the game and our model. The script continuously:

1. Captures the game screen
2. Runs the student model to get steering/throttle/brake
3. Smooths and scales those outputs
4. Sends them into the game via a virtual gamepad

This lets the car operate autonomously in real time.

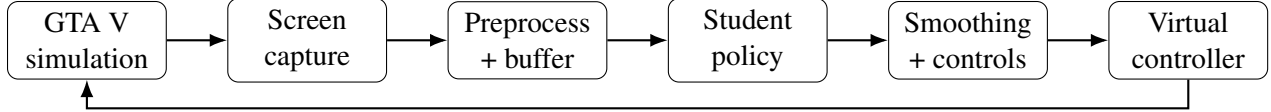


Figure 3: Real-time deployment loop: GTA V renders frames which are captured, preprocessed, and stacked into a small temporal buffer. The student policy predicts steering, throttle, and brake, which are smoothed and sent to the game via a virtual controller, forming a closed control loop.

3.6.1 Screen Capture and Preprocessing

Our agent uses the mss [22] library to capture the GTA V screen in real time to feed the model with frames. The captured frame is first converted into a PIL RGB image, then resized to the correct spec the model is expecting, in our case 260x260, and then scaled $[0, 1]$ using the ImageNet mean and standard deviation. These preprocessing techniques ensure that runtime input matches the training data distribution and is placed into a temporal buffer. This temporal buffer stores the latest 3 frames that allows the model to use the temporal context to perform reasoning during the inference.

3.6.2 Model Inference

The student policy is loaded and run locally on the GPU. The stacked 3 frame tensor is sent through the model and produces 3 outputs as mentioned before: steering in $[-1, 1]$, throttle and braking in $[0, 1]$. The raw predictions from the neural network are inherently not stable from frame to frame and can oscillate wildly, so the agent applies several filters to ensure that stable controls are being sent to the car. We first perform an exponential moving average (EMA) smoothing to the steering and pedal values to ensure that the controls from frame to frame are not significantly different. We then apply a rate-limit to cap how much steering or throttle can change from frame to frame. We do not apply this limit to braking for safety reasons. Every model we trained had some strengths and weaknesses, some with poor acceleration and others with not applying enough steering torque. so we apply gain multipliers on outputs so they feel better in game. Finally, we apply a policy to resolve simultaneous braking and throttling and deadzones to eliminate jitter when either values are insignificant.

3.6.3 Sending Controls to Game

The agent uses a virtual XBOX 360 controller via ViGEmBus and vgamepad Python library. Steering is mapped to the left joystick, throttle is the right trigger, and to avoid reversing when braking we use RB as the handbrake. The driving is not enabled by default. It must be enabled by the user by pressing F4 and will toggle between engaged and manual driving states. the loop runs at the specified frequency for our use case 30 FPS. The complete capture→inference→control cycle while also running the game is able to real time on our realistic RTX 2060 Super, a consumer grade GPU.

4 Experiments

We evaluate the performance of the student-teacher driving system based on both offline metrics from our validation data and live performance in GTA V. The goal is to understand how well the student model can generalize the scene understanding capabilities of modern LLMs and see if it is effective enough to be playable in real time in a simulated environment.

4.1 Offline Evaluation

4.1.1 Performance

Most of the labels in our dataset are of driving straight as is most driving in the real world, so achieving high performance in steering required lots of tuning of the model and the loss function.

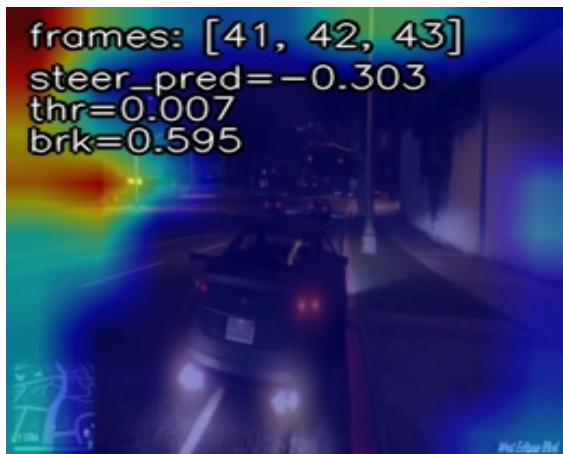
Final results are as followed:

Steer MAE	Throttle MAE	Brake MAE	Turn MAE	Turn Sign Acc
.0583	.1100	.1200	.1400	79%

Steer MAE is the lowest out of the 5 metrics, but a better metric we found to judge steering performance is the turn MAE. Turn MAE measures how off the steering predictions were when the expected steering was greater than 0.1, this is where we see the limitations of the system. Our predictions were 14% off when steering is off, nonetheless still encouraging to see the system this effective with limited data. Another metric we use to judge steering performance is the Turn Sign Acc, this metric simply means when the label has a negative steer value (e.g., turn left) what percent of the time do we actually turn left? 80% is not where we need to be for this to be an actually accurate driving system but it is still encouraging and shows that the model does have understanding of the lane structures. Brake and Throttle MAE being off by 12% on average is actually quite good and shows understanding of when to apply throttle and brakes.

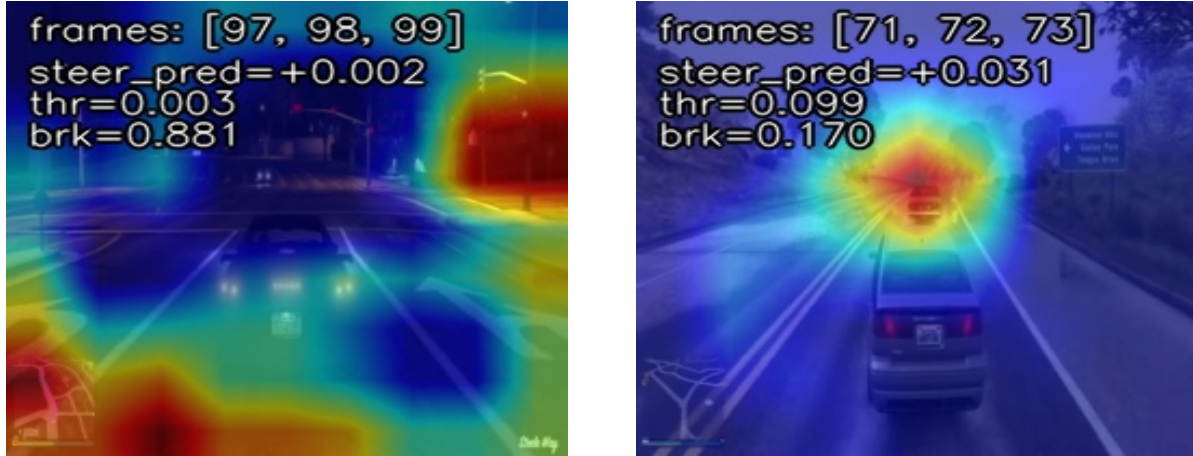


(a) Stopping at the line, where no stop sign exists



(b) Making a sharp left turn to avoid curb and applying brakes

Figure 4: In game examples



(a) Vehicle stopping for a red light, messy activations (b) Clear activations on highway, vehicle is ahead

Figure 5: In game examples

4.2 Online (In-Game) Evaluation

We evaluated the model real time in the game as described earlier. The system operates at 60 FPS with the full control loop operating within 10 ms on the RTX 2060 Super.

In our observations the driving model has a while to go to be a reliable driving agent with highly unreliable steering performance with consistently under-steering and over-steering curves and sometimes not even steering for a curve. The throttle response is quite good but does not understand the context of where to drive fast and when to drive slower, such as in city streets versus on highways. More data will likely solve this issue. Braking has been a significant weakness of the model where crashing into cars ahead and phantom braking is rampant. When it does come to a stop the model shows great hesitancy to continue causing an overall negative user experience. A bright spot is its ability to recognize traffic lights where it is consistent in coming to a stop, although it is not always in the right spot. There have been times when the light turns red and the car is driving through the intersection and suddenly comes to a full stop in the intersection, very unsafe behavior. Highway driving was significantly better than city driving where the model is able to better identify cars ahead and make better steering decisions, as curves are quite gentle on highways. City driving was quite a bit worse with inability to understand the complex intersections, parked vehicles, and more complex lighting conditions. We saw that the driving performance was about the same in all conditions, highlighting the ability of the model to generalize the driving task. I believe with more training data closer to 100 GB up from the 10 GB we currently have, we will greatly improve performance all across the board.

4.2.1 Overall Results

The proposed teacher-student pipeline produces a small yet capable real time vision model using only frame input from the game. While overall performance is yet to be desired, there are definitely some bright spots and provides us with a framework to scale this up in the future, maybe even use this in real vehicles. Nevertheless, we believe we demonstrated that LLM generated driving labels can be distilled into a smaller model with promising real-time behavior.



Figure 6: Images used in training

5 Discussion: Limitations and Future Work

While the self driving agent successfully demonstrates real autonomous capabilities in GTA V, there are several limitations that provide directions for future development.

5.1 Dependence on LLM Teacher Quality

The student’s behavior is fundamentally constrained by the ability for the LLM to fully understand the scene to provide useful labels. LLM outputs are not fully deterministic, so it is possible that the teacher may misinterpret lane structures, inconsistent throttle and braking behaviors, and generating labels that may work in isolation, but given the context of the drive may not be optimal. Some ways to solve these are to use a majority-voting system across many different LLM models to avoid being solely reliant on a single model (GPT-5 Mini)[30][29] like we do in our case.

5.2 Limited High-Level Reasoning

The student model is intentionally small to achieve real-time performance on a consumer grade GPU. As a result, it is able to imitate low-level driving behavior but does not possess the memory beyond the 3 frames that are stacked, it is not able to do long-horizon planning, and how to interact with the non player characters and pedestrians. To solve this, we could look into integrating lightweight transformers and or recurrent modules to better process the frames as a sequence, as our current CNN backbone struggles with.

5.3 Dataset Diversity

In our opinion, dataset quality is the single biggest factor for our performance issues. While highways are well represented in our dataset so, the results are relatively strong compared to city driving. Fully capturing city driving behavior will require significantly more data than we had previously thought, so increasing the city driving clips will be a big priority in our upcoming development. We also need to be able to handle rare scenarios, such as animals running onto roads or objects flying into our car. Also being able to handle for pedestrians is a big priority, as the agent unfortunately caused the deaths of many characters in the game.

5.4 Lack of Explicit Safety Modeling

The model does not explicitly perform collision avoidance, object detection, or any risk estimation based automatic emergency braking systems is a cause for concern with the real world safety and limits the systems ability to handle for catastrophic failures. Some ways to fix this is to train separate models that are specifically designed for lane detection, distance estimates, and automatic emergency braking.

6 Conclusion

We introduce a novel student-teacher pipeline for learning autonomous driving behavior inside a simulated environment. By leveraging an LLM to generate labels as an expert human driver we are able to distill the scene understanding behavior of several billion parameter state of the art LLMs into a small 10 Million parameter model able to run on consumer grade GPUs. We are able to bypass the time-consuming and expensive human annotations and enable quicker development of fully end-to-end vision perception models with surprisingly impressive self-driving behavior when considering the constraints. That being said, there are many limitations of our systems that may impede on the viability of this idea in real self driving cars.

References

- [1] Wen, C., Sun, J., Dong, H., Zhao, T., & Wang, H. (2024). On the road with GPT-4V(ision): Autonomous driving as a case study of multimodal reasoning. arXiv:2310.10044.
- [2] Zhang, F., Chen, Z., Xu, M., & Xie, E. (2023). Think-Driver: Learning driving scene understanding and decision-making from vision-language models. arXiv:2312.00751.
- [3] Fu, Z., Liu, L., Zhang, Z., & Wang, X. (2023). Drive like a human: Rethinking autonomous driving with large language models. arXiv:2308.16855.
- [4] Xu, D., Zhang, J., Zheng, Z., Wang, J., & Wang, Y. (2023). DriveGPT4: Interpretable end-to-end autonomous driving via large language models. arXiv:2312.02264.
- [5] Shao, R., Jin, X., Song, J., & Chen, L. (2024). LMDrive: Closed-loop end-to-end driving with large language models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

- [6] Yang, Y., Liu, S., Li, P., & Wang, M. (2023). A survey of large language models for autonomous driving (LLM4AD). arXiv:2311.12448.
- [7] Li, Z., Chen, H., & Zhang, W. (2025). Applications of large language models and multimodal models in autonomous driving. *IEEE Intelligent Systems*.
- [8] Bojarski, M., Del Testa, D., Dworakowski, D., et al. (2016). End to end learning for self-driving cars. arXiv:1604.07316.
- [9] LeCun, Y., Muller, U., Ben J., et al. (2006). Off-road obstacle avoidance through end-to-end learning. *NIPS*, 18.
- [10] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv:1503.02531.
- [11] Rusu, A. A., Colmenarejo, S. G., Gülçehre, Ç., et al. (2015). Policy distillation. arXiv:1511.06295.
- [12] Singh, A., Kaur, J., & Verma, R. (2023). End-to-end autonomous driving using deep learning: A survey. *Journal of Intelligent Transportation Systems*, 27(2), 134–152.
- [13] Kendall, A., Hawke, J., Janz, D., et al. (2018). Learning to drive in a day. arXiv:1807.00412.
- [14] Pomerleau, D. (1988). ALVINN: An autonomous land vehicle in a neural network. *Advances in Neural Information Processing Systems*, 1, 305–313.
- [15] Tan, M., & Le, Q. (2021). EfficientNetV2: Smaller models and faster training. *Proceedings of the 38th International Conference on Machine Learning (ICML)*. arXiv:2104.00298.
- [16] Howard, A., Sandler, M., Chu, G., et al. (2019). Searching for MobileNetV3. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. arXiv:1905.02244.
- [17] Woo, S., Park, J., Lee, J.-Y., & Kweon, I. S. (2018). CBAM: Convolutional block attention module. *European Conference on Computer Vision (ECCV)*. arXiv:1807.06521.
- [18] Karpathy, A., Toderici, G., Shetty, S., et al. (2014). Large-scale video classification with convolutional neural networks. *CVPR*.
- [19] Sun, X., Tan, L., & Wang, K. (2022). A lightweight motion-aware multi-frame CNN for real-time video understanding. *Pattern Recognition Letters*, 157, 36–44.
- [20] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.
- [21] Girdhar, R., Ramanan, D. (2017). Attentional pooling for action recognition. *NeurIPS*.
- [22] Beltzung, A. (2023). mss: Fast cross-platform multiple screenshots module. <https://github.com/BoboTiG/python-mss>
- [23] Comma.ai. (2023). openpilot documentation. <https://docs.comma.ai>

- [24] iGEmBus. (2024). Virtual Gamepad Emulation Framework. <https://vigem.org>
- [25] Richter, S. R., et al. (2016). Playing for Data: Ground Truth from Computer Games. <https://arxiv.org/abs/1608.02192>
- [26] Radosavovic, I., et al. (2018). Data Distillation: Towards Omni-Supervised Learning. <https://arxiv.org/abs/1712.04440>
- [27] Zhou, X., et al. (2018). Temporal Convolutional Networks for Sequence Modeling. <https://arxiv.org/abs/1803.01271>
- [28] Tremblay, J., et al. (2018). Training Deep Networks with Synthetic Data. <https://arxiv.org/abs/1806.07828>
- [29] Xu, H., et al. (2023). GPT-4V(ision) System Card. OpenAI. <https://openai.com>
- [30] OpenAI. (2023). GPT-4 Technical Report. <https://arxiv.org/abs/2303.08774>